

TASK 3: SQL FOR DATA ANALYSIS

E-Commerce Database Analysis

Objective: Use SQL queries to extract, manipulate, and analyze structured data from a relational database.

Tools: MySQL

Student Name	Arshiya Chaudhary
Course	B.Sc. Data Science
Task	Task 3 – SQL for Data Analysis
Database	E-Commerce SQL Database (MySQL-compatible)

1. Objective

The objective of this task is to learn how SQL can be used for data analysis. The project demonstrates data retrieval with SELECT and WHERE, sorting with ORDER BY, grouping with GROUP BY, aggregate functions, joins, subqueries, views, NULL handling, and basic query optimization using indexes.

2. Dataset and Database Design

A small e-commerce database is used so that all required SQL concepts can be demonstrated with realistic customer, product, order, and order-item relationships.

Table	Important Columns	Purpose
customers	customer_id, customer_name, email, city, signup_date	Customer information
products	product_id, product_name, category, price	Product catalogue
orders	order_id, customer_id, order_date, status	Customer orders
order_items	order_item_id, order_id, product_id, quantity	Products and quantities in each order

Relationship: customers → orders → order_items → products.

3. SQL Database Setup

Run the following SQL first to create the database tables and sample records.

```
-- Create tables
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  email VARCHAR(150),
  city VARCHAR(50),
  signup_date DATE
);

CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  category VARCHAR(50),
  price DECIMAL(10,2)
);
```

```

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date DATE,
    status VARCHAR(20),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Sample customers
INSERT INTO customers VALUES
(1, 'Aarav Shah', 'aarav@example.com', 'Mumbai', '2026-01-05'),
(2, 'Priya Mehta', 'priya@example.com', 'Pune', '2026-01-08'),
(3, 'Rahul Patil', 'rahul@example.com', 'Mumbai', '2026-02-10'),
(4, 'Neha Joshi', 'neha@example.com', 'Nashik', '2026-02-18'),
(5, 'Riya Singh', 'riya@example.com', 'Delhi', '2026-03-01');

-- Sample products
INSERT INTO products VALUES
(101, 'Laptop', 'Electronics', 65000.00),
(102, 'Headphones', 'Electronics', 2500.00),
(103, 'Office Chair', 'Furniture', 8500.00),
(104, 'Keyboard', 'Electronics', 1800.00),
(105, 'Backpack', 'Accessories', 2200.00),
(106, 'Desk Lamp', 'Furniture', 1500.00);

-- Sample orders
INSERT INTO orders VALUES
(1001, 1, '2026-03-01', 'Completed'),
(1002, 2, '2026-03-03', 'Completed'),
(1003, 1, '2026-03-10', 'Completed'),
(1004, 3, '2026-03-15', 'Cancelled'),
(1005, 4, '2026-03-18', 'Completed'),
(1006, 5, '2026-03-20', 'Completed');

-- Sample order items
INSERT INTO order_items VALUES
(1, 1001, 101, 1),
(2, 1001, 102, 2),
(3, 1002, 103, 1),
(4, 1002, 104, 2),
(5, 1003, 105, 1),
(6, 1003, 102, 1),
(7, 1004, 101, 1),
(8, 1005, 106, 2),
(9, 1005, 104, 1),
(10, 1006, 102, 1),
(11, 1006, 105, 2);

```

4. Required SQL Queries and Analysis

4.1 SELECT – Retrieve Data

```
SELECT * FROM customers;
```

Analysis: Displays all customer records from the customers table.

4.2 WHERE – Filter Data

```
SELECT customer_id, customer_name, city
FROM customers
WHERE city = 'Mumbai';
```

Analysis: Returns customers whose city is Mumbai.

4.3 ORDER BY – Sort Data

```
SELECT product_name, category, price
FROM products
ORDER BY price DESC;
```

Analysis: Lists products from highest price to lowest price.

4.4 GROUP BY – Group Records

```
SELECT status, COUNT(*) AS total_orders
FROM orders
GROUP BY status;
```

Analysis: Counts orders for each order status.

4.5 Aggregate Functions – SUM and AVG

```
SELECT
    SUM(p.price * oi.quantity) AS total_revenue,
    AVG(p.price) AS average_product_price
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id;
```

Analysis: Calculates the total value represented by order items and the average product price.

4.6 INNER JOIN – Customers and Orders

```
SELECT c.customer_name, o.order_id, o.order_date, o.status
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id;
```

Analysis: Shows only customers who have matching orders.

4.7 LEFT JOIN – Include Customers Without Orders

```
SELECT c.customer_name, o.order_id, o.order_date
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Analysis: Shows every customer and matching orders; customers without orders still appear.

4.8 RIGHT JOIN – Include All Orders

```
SELECT c.customer_name, o.order_id, o.status
FROM customers c
RIGHT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Analysis: Shows every order and matching customer. Note: SQLite does not support RIGHT JOIN in older/common versions; use the equivalent LEFT JOIN by reversing table order.

4.9 Revenue by Customer

```
SELECT
    c.customer_id,
    c.customer_name,
    SUM(p.price * oi.quantity) AS revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'Completed'
GROUP BY c.customer_id, c.customer_name
ORDER BY revenue DESC;
```

Analysis: Calculates completed-order revenue generated by each customer.

4.10 Subquery – Customers Above Average Product Price

```
SELECT product_name, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
)
ORDER BY price DESC;
```

Analysis: Uses a subquery to find products priced above the overall average product price.

4.11 Subquery – Highest Revenue Customer

```
SELECT customer_name
FROM customers
WHERE customer_id = (
    SELECT o.customer_id
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.status = 'Completed'
    GROUP BY o.customer_id
    ORDER BY SUM(p.price * oi.quantity) DESC
    LIMIT 1
);
```

Analysis: Returns the customer with the highest completed-order revenue.

4.12 HAVING – Filter Groups

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Analysis: Returns customers who have placed more than one order.

4.13 View – Create an Analysis View

```
CREATE VIEW customer_revenue AS
SELECT
    c.customer_id,
    c.customer_name,
    SUM(p.price * oi.quantity) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
```

```

JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'Completed'
GROUP BY c.customer_id, c.customer_name;

SELECT * FROM customer_revenue
ORDER BY total_revenue DESC;

```

Analysis: Creates a reusable virtual table containing customer revenue.

4.14 NULL Handling

```

SELECT customer_id, customer_name,
       COALESCE(email, 'Not Provided') AS email
FROM customers;

```

Analysis: Replaces NULL email values with the text 'Not Provided'.

4.15 Index – Basic Query Optimization

```

CREATE INDEX idx_orders_customer_id
ON orders(customer_id);

CREATE INDEX idx_order_items_order_id
ON order_items(order_id);

CREATE INDEX idx_order_items_product_id
ON order_items(product_id);

```

Analysis: Indexes columns frequently used for joins and filtering. Indexes should be added based on query workload because excessive indexes can slow INSERT/UPDATE operations.

5. Revenue Calculation: Average Revenue Per User

Average Revenue Per User (ARPU) can be calculated by dividing total completed revenue by the number of unique customers who generated that revenue.

```

SELECT
    SUM(p.price * oi.quantity) /
    COUNT(DISTINCT o.customer_id) AS average_revenue_per_user
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'Completed';

```

This query first calculates revenue from completed orders and then divides it by the number of unique customers with completed orders.

6. Expected Analysis / Findings

- SELECT and WHERE help retrieve only the records required for analysis.
- ORDER BY makes it easy to identify high-priced products or top customers.
- GROUP BY with COUNT, SUM, and AVG converts transaction-level data into useful business summaries.
- INNER JOIN combines matching customer, order, order-item, and product information.
- LEFT JOIN is useful when all customers must be retained, including customers with no orders.
- Subqueries allow comparisons against calculated values such as the average product price.
- Views provide reusable datasets for dashboards and repeated analysis.
- Indexes can improve joins and filters on frequently queried columns.

8. Questions and Answers

1. What is the difference between WHERE and HAVING?

WHERE filters individual rows before GROUP BY and aggregation. HAVING filters grouped results after GROUP BY. For example, WHERE can select completed orders, while HAVING can select customers whose order count is greater than 1.

2. What are the different types of joins?

The main joins are INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN. INNER JOIN returns matching rows; LEFT JOIN returns all rows from the left table plus matches; RIGHT JOIN returns all rows from the right table plus matches; FULL OUTER JOIN returns all rows from both tables with NULLs where no match exists. CROSS JOIN returns combinations of every row.

3. How do you calculate average revenue per user in SQL?

Calculate total revenue from the relevant transactions and divide it by the number of unique users/customers. A typical expression is `SUM(revenue) / COUNT(DISTINCT user_id)`. In an e-commerce database, revenue can be calculated as `price × quantity`.

4. What are subqueries?

A subquery is a query written inside another SQL query. It can be used in SELECT, FROM, or WHERE/HAVING clauses. For example, a subquery can calculate the average product price and the outer query can return products above that average.

5. How do you optimize a SQL query?

Use appropriate indexes, select only required columns instead of `SELECT *`, filter early, avoid unnecessary joins, use efficient join conditions, inspect the execution plan with EXPLAIN, and avoid functions on indexed columns when possible. Indexes should be created based on actual query patterns.

6. What is a view in SQL?

A view is a virtual table based on a stored SQL query. It normally stores the query definition rather than a separate copy of the data. Views simplify repeated analysis, improve readability, and can provide controlled access to selected columns or rows.

7. How would you handle NULL values in SQL?

Use `IS NULL` or `IS NOT NULL` to test for NULL. Use `COALESCE()` to replace NULL with a default value, and use `NULLIF()` when appropriate. Aggregates such as SUM and AVG generally ignore NULL values, but the analysis should still consider whether missing data changes the business result.

9. Conclusion

This task demonstrated how SQL can be used for practical data analysis in an e-commerce environment. The project covered data extraction, filtering, sorting, grouping, aggregate calculations, joins, subqueries, views, NULL handling, and indexes. These techniques form an important foundation for data analyst and business intelligence roles because they allow analysts to transform relational data into meaningful business insights.